TinySoC spec

1.0.0

Date: 2026/10/18

Tensor Accelerator SoC

浙江大学

Revision History

Revision	Date	Description	Author
1.0.0	2026-10-19	首次定义 TinySoC 验证平台的架构、地址映射、VTA 编程模型、接口和执行规则。	Wangshu

文档对象

本规格面向 TinySoC 的 RTL 集成、固件开发、验证与移植。平台通过 RISC-V CPU 配置张量加速器，以共享存储和 AXI DMA 完成用例执行，并由 CPU 检查输出结果。

适用版本

基线为 tinysoc_wangshu 当前 APB32 / AXI64 仿真架构。NPU User Guide V1.1 对应 8×8 NPU；本地冻结 RTL 为 16×16 实例。两种实例的公共集成接口在本文中统一描述，配置差异在 1.3 节及附录 B 中单独定义。

版本管理规则

SoC 配置、NPU RTL、端口映射、存储封装和用例数据须作为同一发布配置核对。VTA 指令集编码、张量分块或内部 Buffer 容量变化时，应重新生成或确认配套用例，不能仅以外部端口相同判定兼容。

目录

1 Introduction....4
1.1 Overview....4
1.2 Feature....4
1.3 配置与适用范围....4
1.4 术语与参考规则....5
2 功能描述....6
2.1 TinySoC 框图....6
2.2 CPU 与系统互连....6
2.3 VTA 功能模块....7
2.4 存储子系统....8
2.5 时钟与复位....8
2.6 固件与镜像....8
3 寄存器....9
3.1 REGS Base Address....9
3.2 CSR Bank Map....9
3.3 VTA Register Map....9
3.4 Register Description....10
3.5 寄存器访问规则....12
4 TinySoC 架构与接口....13
4.1 模块连接....13
4.2 时钟与复位接口....13
4.3 APB 控制接口....13
4.4 AXI DMA 接口....14
4.5 UART 与仿真辅助接口....15
4.6 存储实现接口....15
4.7 RTL 包与扩展接口....15
5 通信规则....16
5.1 CPU 与 NPU 执行流程....16
5.2 DMA 地址与缓存规则....16
5.3 AXI Burst 拆分规则....17
5.4 状态与异常处理....17
5.5 用例与结果输出....18
5.6 构建与验证规则....18
5.7 移植与启动规则....19
附录 A 参考文件与运行入口....20
A.1 服务器环境与执行....20
附录 B 配置差异与扩展边界....21
B.1 当前本地冻结包标识....21

1 Introduction

1.1 Overview

TinySoC 是一个可扩展张量加速器的 SoC 验证平台。当前以 VexRiscv RV32 为主机、VTA 为 NPU，通过 LiteX / Migen 构建 CPU、AXI 互连、Boot ROM、共享 SRAM、DDR3 验证模型和 UART，使用 Verilator 执行 SoC 级验证。

CPU 经系统互连和 Wishbone 至 APB 桥访问 NPU 控制寄存器；NPU 通过独立 AXI Master 端口获取指令与张量数据，并将结果写回共享存储。CPU 负责配置、完成轮询、缓存维护和结果比较，数值计算由 NPU 完成。

本文定义系统集成和软件可见行为。张量指令、uOp 编码及算子编译策略由配套 NPU 实现和离线生成工具定义；它们不由 SoC 接口自动映射工具推导。

1.2 Feature

- 单 CPU 加单 NPU，32 位系统地址空间，系统 AXI 数据通路为 32 位。
- APB4 32 位控制接口，当前 NPU AXI 数据口为 64 位，地址为 32 位，ID 为 8 位。
- Boot ROM 128 KiB、共享 SRAM 128 KiB、DDR 160 KiB 精确地址窗口；共享 SRAM 可被 CPU 和 NPU DMA 访问。
- 支持 2D LOAD、STORE、GEMM 与 Tensor ALU 的系统级验证；8×8 手册实例支持 INT8×INT8 及 INT32 累加。
- 支持独立 ROM、SRAM、DDR 镜像预加载；仿真启动时不要求 CPU 复制用例初始化数组。
- 冻结 RTL 包采用 SHA256 和语义端口映射校验，支持顶层改名、端口映射及复位极性声明。
- 提供 C 用例驱动、UART PASS / FAIL 输出和宿主退出码，支持离线 CentOS 7 运行环境。

1.3 配置与适用范围

项目	NPU 手册与正式集成配置	本地冻结 RTL 基线
计算并行度	8×8，手册峰值 64 MAC / cycle	16×16，LOG_BLOCK=4
张量位宽	INT8 输入与权重，INT32 累加	LOG_INP_WIDTH=3， LOG_WGT_WIDTH=3， LOG_ACC_WIDTH=5
内部存储	容量及宏封装随正式 RTL 发布	当前 vta.v 使用行为存储数组
配套用例	bc_npu 及其正式数据头	dense_gemm_bias_clip、 mlperf_resnet8_qconv
控制与 DMA	按正式顶层声明映射	APB32 / AXI64

8×8 手册实例的 64 MAC / cycle 是算术阵列峰值，不是整机吞吐量。200 MHz 为当前仿真配置；本文不把它声明为 FPGA 或 ASIC 的已收敛工作频率。内部 Buffer 容量须从对应 RTL 或生成配置核实，不能由 SoC 共享 SRAM 容量替代。

当前交付目标为仿真平台。FPGA 板级 target、真实 DDR PHY、bitstream 流程、完整综合约束和专业 CDC / RDC 签核均属于扩展集成范围。当前不包含操作系统、MMU / IOMMU、L2 Cache、LiteX 定时器或已连接的 NPU 中断服务。

1.4 术语与参考规则

术语	定义
VCR	VTA Control Registers，NPU 软件可见控制寄存器。
VME	VTA Memory Engine，对内部访问请求进行 AXI 访存和仲裁。
uOp	微操作，描述 GEMM / ALU 的局部索引与计算操作。
共享 SRAM	SoC 地址空间中的 128 KiB RAM，与 NPU 内部 Buffer 分开。
golden	用于检查 NPU 输出的期望结果数据。
冻结 RTL 包	预生成 RTL、manifest、filelist、配置及端口映射组成的交付单元。

地址、端口连接和寄存器实际行为以当前配置及 RTL 为准；NPU 手册用于解释模块功能与编程模型。手册与当前实现不同的项目按附录 B 管理。参考文件清单见附录 A。

2 功能描述

2.1 TinySoC 框图

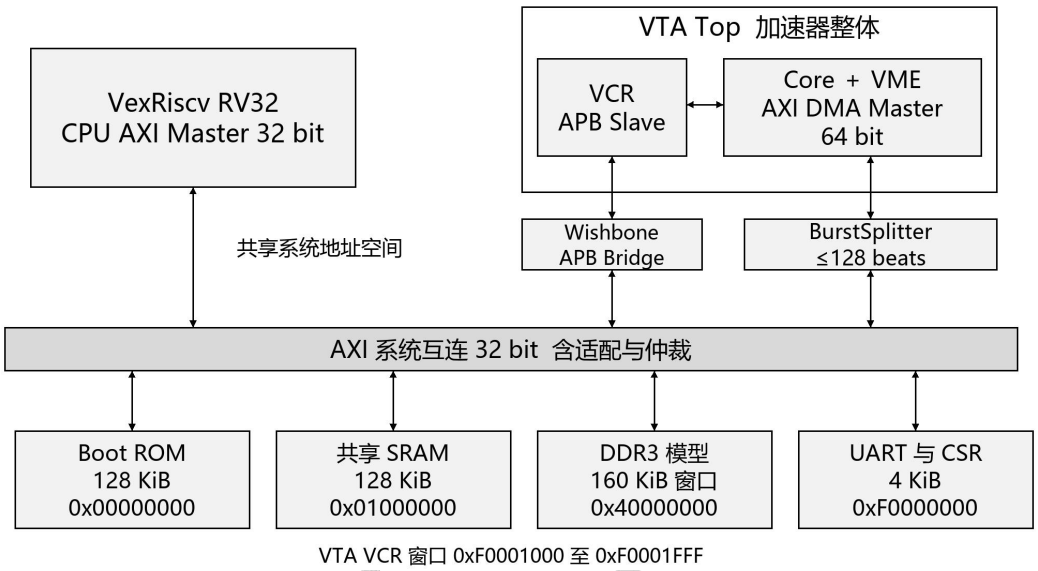


Figure 2-1 TinySoC 系统架构

VTA 在架构上作为一个完整 NPU 模块，其 APB Slave 控制口与 AXI Master 数据口承担不同职责。图中的桥和 BurstSplitter 属于 SoC 适配逻辑，不是另一颗 NPU。CPU 与 NPU 可共享 SRAM 和 DDR，但不提供硬件缓存一致性协议。

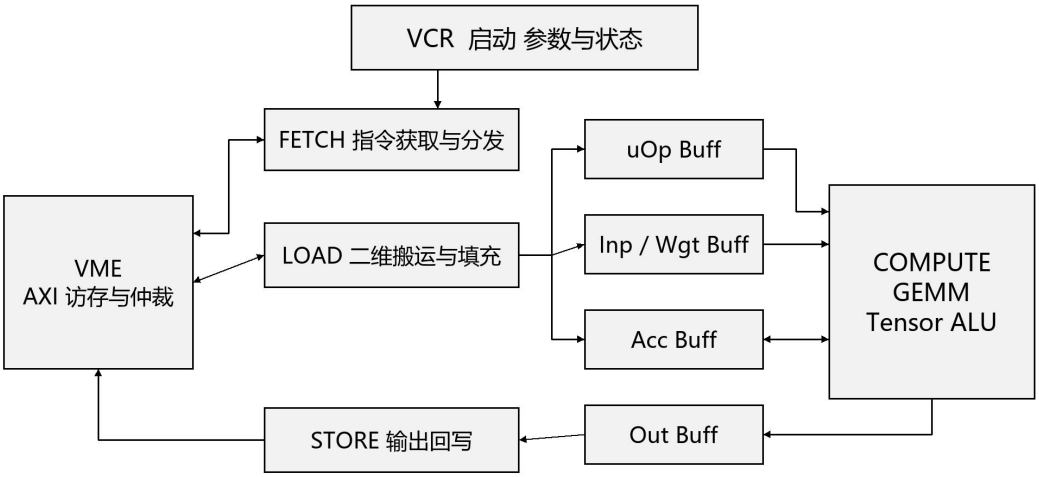
2.2 CPU 与系统互连

当前 CPU 为 VexRiscv standard 变体，运行裸机用例固件。SoC 系统互连采用 AXI，数据宽度 32 位、地址宽度 32 位。NPU 的 64 位 AXI DMA 通过长突发拆分和下游位宽适配加入互连。总线仲裁及宽度转换由 LiteX 集成实现。

NPU VCR 以非缓存设备窗口注册到系统总线。CPU 的一次寄存器访问经 LiteX 总线适配到 Wishbone，再由 WishboneToAPB32 完成 APB SETUP / ACCESS 周期。当前桥同一时间处理一笔控制请求，并传播 APB 错误为 Wishbone err。

当前 SoC 不包含 L2 Cache。配置中的 cached=false 是地址映射属性，不能代替 CPU 实际缓存策略确认；DMA 前后的缓存维护按 5.2 节执行。

2.3 VTA 功能模块



NPU 局部 Buffer 与 SoC 共享 SRAM 分开

Figure 2-2 VTA 功能组成

NPU 手册描述的 8×8 实例由控制寄存器、Memory Engine、FETCH、LOAD、STORE、GEMM 和 ALU 组成。LOAD、COMPUTE、STORE 支持任务级并行，其依赖通过指令中的同步机制协调。离线指令流应显式保证生产和消费顺序。 [1，第 1 至 2 页]

模块	主要功能	系统交互
VCR	保存启动状态、指令数量和六个数据基址；锁存完成统计。	CPU 经 APB 读写。
FETCH	按 InsNum 和 InsBaddr 获取指令，分发给 Load / Compute / Store。	经 VME 发起 AXI 读。
LOAD	将 uOp、输入、权重及累加初值搬入内部 Buffer，支持二维数据与常量填充。	从共享 SRAM / DDR 读取。
GEMM	读取 uOp、输入和权重，执行矩阵乘加并更新累加数据。	在 NPU 局部 Buffer 内计算。
Tensor ALU	执行 ADD、MIN、MAX、移位及乘法类操作，支持量化处理。	具体编码及舍入由配套 RTL 定义。
STORE	将 Out Buff 数据写回共享地址空间。	经 VME 发起 AXI 写。
VME	汇集内部客户端读写请求，产生和接收 AXI 事务。	外接 BurstSplitter 与系统互连。

Conv2D、Dense、ReLU、ADD 和 MUL 属于手册列出的算子能力，实际由配套编译或指令生成流程映射到硬件指令。当前清版 SoC 包消费现成指令和数据数组，不包含完整 TVM 模型编译链。

2.4 存储子系统

2.4.1 Boot ROM

Boot ROM 保存 RISC-V 启动代码、用例程序和普通只读常量。复位入口按当前 CPU 与 ROM 基址配置生成。它不是 NPU 输出存储区。ROM 可保存 DDR 初始化所需的固件逻辑；DDR 存储本身由独立控制器和模型提供。

2.4.2 共享 SRAM

共享 SRAM 是挂在系统总线上的 128 KiB RAM。普通算子用例的指令、uOp、输入、权重、累加初值和输出优先放入该区域，以缩短 NPU DMA 访问延迟。普通可写数据、BSS 和栈也使用此区域，链接时须为它们预留空间。

2.4.3 DDR

DDR 使用 LiteDRAM 的 SDRAMPHYModel 及 MT41K256M8 DDR3 模型，PHY 数据配置为 8 位。SoC 仅开放 160 KiB 的可访问窗口，窗口大小不代表该 DDR 器件的全部物理容量。ExactSizeSoCRegion 对边界作精确译码，避免将 160 KiB 自动扩大为 256 KiB。

普通算子用例的 golden 数据放 DDR；bc_npu 使用专用链接脚本，将初始化工作集及普通可写数据放入 DDR，BSS 和栈也放 DDR。实际容量必须由 ELF / map 文件核对，不能假设模型规模任意增大后仍能装入。

2.5 时钟与复位

当前平台配置为单一 sys 时钟域，CPU 与 NPU 使用同一系统时钟，频率配置为 200 MHz。FrozenVTA 将主 clock 和可选 host.pclk 都绑定至 sys。当前本地 VTA 顶层 reset 为高有效，同步时序逻辑在 clock 上升沿采样；替换 RTL 时可在 interface.json 中声明低有效复位并由封装反相适配。

复位会清零 VCR 状态、基址和计数快照，但不应假设复位会清空全部 SRAM、DDR 或 NPU 内部数组。仿真数据的初值来自镜像或行为模型。实际多时钟、时钟门控及异步复位释放须另行定义 CDC / RDC 约束。

2.6 固件与镜像

镜像或段	普通算子用例	BC 用例
bios.bin	ROM 程序和普通只读内容	ROM 程序和普通只读内容
sram.bin	来自 .vta_data 的工作集及普通初始化可写数据	一字零值占位镜像
ddr.bin	来自 .expected_data 的 golden 数据	来自 .vta_data 的初始化数据
BSS 与栈	SRAM；BSS 由启动代码清零，栈用于软件运行	DDR；BSS 由启动代码清零，栈用于软件运行

构建分两阶段：先编译用例 ELF 并提取三个镜像，再重新实例化 SoC，把镜像内容装入 ROM、SRAM 和 DDR 仿真模型。链接脚本保持初始化数据的装载地址与运行地址一致，并使 CPU 的初始化数据复制范围为空；BSS 清零不属于用例数组复制。

3 寄存器

3.1 REGS Base Address

系统地址均为字节地址，端点采用包含关系表示。VTA VCR_BASE = 0xF0001000，窗口大小 0x1000。未列出的系统区间不属于本配置的有效存储或设备窗口。

区域	Base	End	Size	用途
Boot ROM	0x00000000	0x0001FFFF	128 KiB	CPU 启动与用例程序
SRAM	0x01000000	0x0101FFFF	128 KiB	CPU 与 DMA 共享工作集
DDR	0x40000000	0x40027FFF	160 KiB	main_ram 与 golden / BC 数据
UART / CSR	0xF0000000	0xF0000FFF	4 KiB	四个 1 KiB CSR bank
VTA VCR	0xF0001000	0xF0001FFF	4 KiB	NPU 控制寄存器

UART / CSR 与 VTA VCR 窗口相邻。ROM、SRAM、DDR 之间的地址间隔为当前架构配置，不表示存在连续映射。当前配置校验器固定上述容量；修改容量需要同时调整校验、链接布局、镜像及验证。

3.2 CSR Bank Map

Bank	Base	End	功能
0	0xF0000000	0xF00003FF	UART 仿真通信
1	0xF0000400	0xF00007FF	LiteDRAM 控制与初始化
2	0xF0000800	0xF0000BFF	SimFinish，仅用例仿真存在
3	0xF0000C00	0xF0000FFF	SimMarker，仅用例仿真存在

各 bank 内的寄存器偏移、位宽及访问函数由 LiteX 生成的 csr.csv 和 generated/csr.h 定义。本文不把某次构建的内部 CSR 偏移固定成跨版本 ABI。独立架构生成不实例化 SimFinish / SimMarker。

3.3 VTA Register Map

VCR 使用 32 位寄存器和 4 字节对齐偏移，全部复位值为 0。下表 Access 表示软件使用约定；本地 RTL 允许配置寄存器读回，且计数快照也存在软件写通路，这与手册的 R / W 标签有所不同。软件应按约定使用，不依赖额外写通路。 [1，第 3 至 6 页；4，VCRRegisters]

Offset	Register	Access	Address	Description
0x00	Ctrl	R/W	0xF0001000	启动与完成状态
0x04	CycCnt	R	0xF0001004	完成时锁存的运行周期
0x08	InsNum	W	0xF0001008	指令条数
0x0C	InsBaddr	W	0xF000100C	指令流基址
0x10	UopBaddr	W	0xF0001010	uOp 基址
0x14	InpBaddr	W	0xF0001014	输入基址
0x18	WgtBaddr	W	0xF0001018	权重基址
0x1C	AccBaddr	W	0xF000101C	累加初值基址
0x20	OutBaddr	W	0xF0001020	输出基址
0x24	AccWrCnt	见说明	0xF0001024	累加写事件计数快照

AccWrCnt 在手册中标作 W；本地 RTL 在 finish 时把内部累加写事件计数锁存到该寄存器，且可读回。本文将其作为本地实现的只读统计使用；正式 ASIC 版本的该项语义须随 RTL 发布核实。

3.4 Register Description

3.4.1 Ctrl Register

Offset = 0x00　Address = 0xF0001000　Size = 32 bits

Bits	Field	R/W	Reset	Description
31:2	Reserved	R/W	0	软件写 0；本地 RTL 为存储位，不定义功能。
1	DONE	R/W	0	完成时硬件置 1；运行状态编码为 0x1，完成编码为 0x2。不是 W1C 位。
0	START	R/W	0	软件写 1 启动；该位直接驱动 launch 电平。finish 时硬件清除此位。

软件只使用 0x0 空闲、0x1 执行、0x2 完成三个值；0x3 不定义为有效运行状态。finish 与软件写同周期时，硬件完成更新优先，整个 Ctrl 写为 0x2。启动下一任务使用完整字写 0x1，以同时清除旧 DONE。写 Ctrl=0 不等价于复位 NPU 内部状态，不能作为已定义的中止操作。

3.4.2 CycCnt Register

Offset = 0x04 Address = 0xF0001004 Size = 32 bits

Bits	Field	R/W	Reset	Description
31:0	CycCnt	R	0	finish 时锁存 cycle_cnt，运行期间不实时更新。

内部计数器在 launch=1 且 finish=0 时每拍加 1，其他条件清零；VCR 在 finish 时保存其值。完成前读取可能得到复位值或上一次结果。CycCnt=0 不能单独区分未启动、尚未完成或错误停滞。软件不得在运行期间写该寄存器。

3.4.3 InsNum Register

Offset = 0x08 Address = 0xF0001008 Size = 32 bits

Bits	Field	R/W	Reset	Description
31:0	InsNum	W	0	本次执行的 NPU 指令数量，单位为条，不是字节。

本地用例按每条 128 bit 指令计算 InsNum，即指令数组字节数除以 16。应在启动前配置非零值；指令数组范围必须完整落入 DMA 可读内存。指令数量与结束指令、依赖同步必须匹配。

3.4.4 InsBaddr Register

Offset = 0x0C Address = 0xF000100C Size = 32 bits

Bits	Field	R/W	Reset	Description
31:0	InsBaddr	W	0	指令数组基址，使用系统字节地址。

指令数组的布局必须符合对应 RTL 的指令与张量分块格式。当前用例将指令数组按 128 字节对齐；128 bit 指令格式不能据此推导所有 DMA 缓冲区都必须按 128 字节对齐。

3.4.5 UopBaddr Register

Offset = 0x10 Address = 0xF0001010 Size = 32 bits

Bits	Field	R/W	Reset	Description
31:0	UopBaddr	W	0	微操作数组基址，使用系统字节地址。

微操作数组采用对应 RTL 的分块格式；当前通用用例通常按 8 字节对齐。

3.4.6 InpBaddr Register

Offset = 0x14 Address = 0xF0001014 Size = 32 bits

Bits	Field	R/W	Reset	Description
31:0	InpBaddr	W	0	输入张量基址，使用系统字节地址。

输入张量采用对应 RTL 的分块格式；当前通用用例通常按 8 字节对齐。

3.4.7 WgtBaddr Register

Offset = 0x18 Address = 0xF0001018 Size = 32 bits

Bits	Field	R/W	Reset	Description
31:0	WgtBaddr	W	0	权重张量基址，使用系统字节地址。

权重张量采用对应 RTL 的分块格式；当前通用用例通常按 8 字节对齐。

3.4.8 AccBaddr Register

Offset = 0x1C Address = 0xF000101C Size = 32 bits

Bits	Field	R/W	Reset	Description
31:0	AccBaddr	W	0	累加初值基址，使用系统字节地址。

累加初值采用对应 RTL 的分块格式；当前通用用例通常按 8 字节对齐。

3.4.9 OutBaddr Register

Offset = 0x20 Address = 0xF0001020 Size = 32 bits

Bits	Field	R/W	Reset	Description
31:0	OutBaddr	W	0	输出张量基址，使用系统字节地址。

输出张量的布局必须符合对应 RTL 的指令与张量分块格式。目标必须可写且 NPU DMA 可达；完成后 CPU 在读取输出前执行缓存失效处理。

3.4.10 AccWrCnt Register

Offset = 0x24 Address = 0xF0001024 Size = 32 bits

Bits	Field	R/W	Reset	Description
31:0	AccWrCnt	R	0	finish 时锁存 io_acc_wr_event 事件计数。

该值统计 NPU 内部累加写事件，不能直接当作 AXI 写事务数、MAC 次数或输出元素数。本地硬件更新优先于软件写；正式 ASIC 的访问权限和更新语义以正式 RTL 为准。

3.5 寄存器访问规则

所有寄存器按偏移精确匹配，软件采用对齐的 32 位读写。APB4 PSTRB 每一位控制相应字节写入；被屏蔽字节保留旧值。Ctrl 的硬件完成更新不受软件 PSTRB 控制。配置寄存器在运行时没有硬件写保护，软件必须在 START 之后禁止改写任务参数。

所有基址在 START 前写入，执行过程中保持不变；本地 RTL 支持配置寄存器读回供调试。可读回不改变其软件配置用途。指令、uOp 和张量的格式及对齐要求应与实际 IP 变体一致。

在当前本地顶层，PSLVERR 固定为 0。VCR 窗口内未定义偏移读回 0、写入不更新任何寄存器；不能依靠这类访问触发异常。对窗口以外地址的响应属于 SoC 互连和从设备行为，本文不保证其返回 0 或自动超时。

4 TinySoC 架构与接口

4.1 模块连接

连接	方向与宽度	功能
CPU → 系统互连	AXI Master；32 位地址与数据	访问 ROM、SRAM、DDR 和 CSR。
系统互连 → VCR 桥	Wishbone 32 位 → APB4 32 位	把系统地址转换为 VCR 局部字节偏移。
VTA → BurstSplitter	AXI Master；64 位数据	保留上游事务语义并拆分长 INCR burst。
BurstSplitter → 系统互连	64 位 AXI，后续由 LiteX 适配	连接共享 SRAM 和 DDR 等目标。
CPU → UART	LiteX CSR → 仿真串行流	输出用例地址、计数与 PASS / FAIL。

4.2 时钟与复位接口

Signal	Direction	Width	Description
clock	I	1	NPU 主时钟，当前绑定 sys。
reset	I	1	当前本地高有效同步复位；低有效变体由接口映射声明。
host.pclk	I 可选	1	正式顶层可能提供 APB 时钟；当前封装将其也接 sys。

下述方向均相对 VTA 顶层。interface.json 保存语义角色至实际端口名的映射，端口名不是跨变体不变的 ABI。当前本地 RTL 无独立 APB 时钟输入；正式顶层若同时出现 clock 与 io_p_clk，需要显式选择主时钟。

4.3 APB 控制接口

Signal role	Direction	Width	Description
PADDR	I	16	局部字节地址；系统仅映射低 4 KiB 窗口。
PSEL	I	1	外设选择。
PENABLE	I	1	访问阶段指示。
PWRITE	I	1	1 为写，0 为读。

Signal role	Direction	Width	Description
PWDATA	I	32	寄存器写数据。
PSTRB	I 可选	4	本地支持逐字节写屏蔽。
PPROT	I 可选	3	桥输出 0；本地 VTA 不使用。
PRDATA	O	32	寄存器读数据。
PREADY	O	1	当前 RTL 在 PSEL 与 PENABLE 同时为 1 时应答。
PSLVERR	O	1	当前本地固定 0；桥可传播其他变体的错误。

WishboneToAPB32 在 IDLE 锁存地址、写方向、数据和字节使能，经 SETUP 拉高 PSEL，再于 ACCESS 拉高 PENABLE。等待 PREADY 时请求字段保持稳定。本地 VCR 无额外等待拍；集成其他 IP 时桥允许延长 ACCESS。

手册架构图标注 APB 3.0，特性列表中的 AXI4 指向其访存能力；TinySoC 实际控制接口按当前 manifest / RTL 的 APB4 32 位定义。协议改变需要桥实现配套修改，端口自动识别不完成协议转换。

4.4 AXI DMA 接口

Channel	NPU 输出	NPU 输入	关键宽度
AW	VALID ADDR ID LEN SIZE BURST 等	READY	ADDR=32，ID=8，LEN=8
W	VALID DATA STRB LAST	READY	DATA=64，STRB=8
B	READY	VALID RESP ID	RESP=2，ID=8
AR	VALID ADDR ID LEN SIZE BURST 等	READY	ADDR=32，ID=8，LEN=8
R	READY	VALID DATA RESP LAST ID	DATA=64，RESP=2，ID=8

每个 AXI 通道以 VALID 与 READY 同时为 1 表示一次传输；VALID 已置位而 READY 为 0 时，源端应保持有效载荷不变。AxLEN 编码为拍数减 1，8 位可表达 1 至 256 拍。BurstSplitter 适用 INCR burst，不能作为通用 FIXED / WRAP 转换器。

本地 Chisel 顶层还暴露 WID、USER、REGION 及 2 位 LOCK 等兼容性信号。当前 LiteX 封装丢弃输出 WID / USER / REGION，将输入 BUSER / RUSER 置 0，并对 LOCK 等宽度适配。该集成不承诺 AXI USER 属性、独占访问或多主缓存一致性。

当前 SoC 侧 NPU DMA ID 宽度固定为 8 位、地址宽度固定为 32 位。工具允许识别其他宽度，不表示整条系统路径已验证支持任意宽度；扩展时须检查接口、拆分器和下游转换的一致性。

4.5 UART 与仿真辅助接口

Signal	Direction	Width	Description
serial.source_valid / ready / data	输出流握手	1 / 1 / 8	SoC 向 serial2console 发送字符。
serial.sink_valid / ready / data	输入流握手	1 / 1 / 8	serial2console 向 SoC 输入字符。
SimFinish / SimMarker	CSR	生成定义	供仿真退出和标记，不是 FPGA 实体引脚。

当前 UART 为字节流仿真接口，不能直接当作板级 TX / RX 管脚使用。FPGA 版本应选择真实 UART PHY、波特率和板载串口引脚；测试程序可继续复用 printf 输出。

4.6 存储实现接口

SoC 共享 SRAM 与 NPU 局部 Buffer 必须分开实现和计量。正式 ASIC VTA 的 mem_wrap 封装与 mem_sim 行为模型是成对交付关系；当前清版 filelist 保留六个正式封装路径，实体文件由正式代码包提供。

FPGA 版本需要把 ASIC SRAM 宏替换为 BRAM 或其他可综合 RAM，保持地址深度、数据宽度、读延迟、写使能、读写冲突和复位行为。DDR 模型须替换为真实控制器 / PHY，或在系统顶层引出适当的内存访问边界。

4.7 RTL 包与扩展接口

File	职责
rtl/vta.v	冻结 NPU RTL，当前顶层 VTA_Top。
filelist.f	按 IP 目录解析的 HDL 源文件清单。
manifest.json	协议、宽度、顶层、来源标识及文件哈希。
interface.json	实际端口、语义角色、复位极性及显式覆盖。
vta_config.json	配套 VTA 配置；须与实际内部结构及用例一致。
rtl.sha256	RTL 校验清单。

新增输入必须显式连接或声明常量，未使用输出必须显式确认；不得静默丢弃新增功能信号。reset_n / rst_n 等常用名称可识别为低有效，非标准名称需显式声明。变更后先同步 RTL 元数据，再提取并校验接口。

当前扩展点是冻结 IP 的替换和接口适配，尚未实现任意协议、多 NPU、多时钟及任意内存容量的一键配置。FPGA bitstream 流程还需板卡 target、引脚和时序约束、PLL / reset、数据装载和实现工具。

5 通信规则

5.1 CPU 与 NPU 执行流程

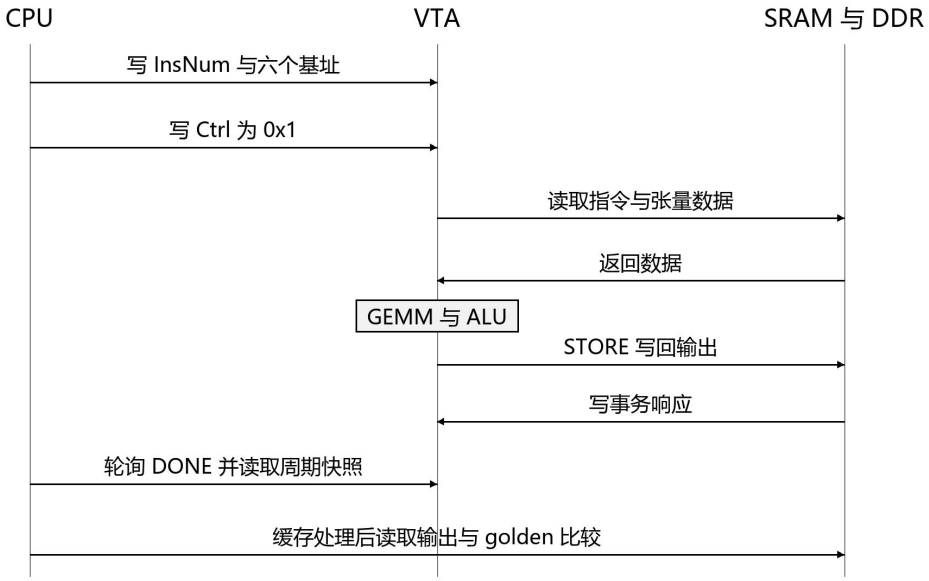


Figure 5-1 轮询模式执行与数据访问

1. 确认 RTL、用例、镜像和地址布局匹配，DDR 或 SRAM 数据已就绪。仿真由模型预加载；硬件由初始化与下载流程准备。

2. 读取 Ctrl，START=1 时禁止提交新任务。对指令、uOp、输入、权重、累加及输出缓冲区执行必要的 cache clean 和内存屏障。

3. 写入 InsNum 与 InsBaddr、UopBaddr、InpBaddr、WgtBaddr、AccBaddr、OutBaddr，最后执行屏障并写 Ctrl=0x1。

4. NPU 根据指令数量和指令基址取指，通过 VME 读取张量，执行 GEMM / ALU，并由 STORE 写回输出。

5. CPU 轮询 Ctrl 的 DONE 位。NPU finish 时，VCR 更新 Ctrl=0x2，并锁存 CycCnt 与本地实现的 AccWrCnt。

6. CPU 在读取输出前完成 cache invalidate 与屏障，然后把输出与 golden 比较，输出 PASS / FAIL。

完成关系还依赖指令流中的同步和终止指令。用例应保证所需 STORE 和写响应已纳入任务完成条件，不能只根据某次计算单元的内部结束信号推定任意输出都已经全局可见。

5.2 DMA 地址与缓存规则

默认 vta_case_dma_address 将 CPU 指针转换为 32 位整数，适用于当前统一物理地址布局。目标平台若存在总线地址偏移、CPU 别名、虚拟地址或 DMA 受限窗口，必须重写该 weak

hook。所有指令和数据范围均须落在 DMA 可达内存，输出范围须可写，地址加长度不得溢出 32 位空间。

vta_case_cache_clean / invalidate 在 LiteX 仿真配置下调用 CPU D-cache 处理函数，并在前后设置屏障。实现使用全缓存维护而非精确范围操作；移植时须按 CPU 缓存实现验证 clean、invalidate 和写回顺序。仅调用编译器屏障不能替代真实缓存维护。

START 后 CPU 不得改写指令或输入缓冲区，也不得与 NPU 并发写输出。默认运行时不提供任务队列、内存保护或数据竞争检测。若要重叠 CPU 与 NPU 工作，应另外定义缓冲区所有权和交接协议。

5.3 AXI Burst 拆分规则

当前 max_beats=128。拆分器为读、写分别设置状态机，每个方向一次接收一笔上游事务；一笔长事务的多个下游片段按顺序发出。读和写方向可独立推进，但本模块不提供同方向多个未完成事务并发。

读路径按剩余拍数生成片段，地址按片段拍数乘每拍字节数递增；只在原始事务的最后一拍向上游置 RLAST。写路径在每个片段结尾产生下游 WLAST，等待该片段 B 响应后再发下一片段，最终只向上游返回一次 B 响应。

写响应保留观察到的非 OKAY 状态；若多个片段出错，当前实现保存最后一次非 OKAY 响应，不定义错误严重性优先级。读响应逐拍转发。拆分器不提供超时、中止恢复或独立的 4 KiB 跨界修复；上游必须提供符合 AXI 边界规则的合法 INCR 事务。

128 拍限制针对当前 64 位 NPU 数据到 32 位系统数据的集成选择。扩大 NPU 数据宽度时，应重新计算下游拍数与 AxLEN 上限，不能直接沿用该常量。

5.4 状态与异常处理

状态或现象	定义与处理
Ctrl=0x0	VCR 空闲；结合启动配置和复位状态判断是否已经执行。
Ctrl=0x1	START 已置位且尚未完成。可能等待存储、依赖或执行结束，单凭此值不能定位 AXI 故障。
Ctrl=0x2	硬件报告任务完成。读取统计、处理缓存并校验输出。
轮询超时	通用运行时返回 -3；保留地址、Ctrl 和 AXI 请求 / 响应信息用于定位。
CPU 总线访问阻塞	软件轮询次数上限未必能生效；需要总线超时或宿主超时机制。
Flash 启动不完成	检查 DDR 初始化、镜像搬运、LMA / VMA、DMA 地址、cache 与 AXI 波形。

通用运行时默认最多轮询 100000000 次，单位为寄存器读取次数，不是 NPU 时钟周期。超时后不自动复位 NPU，也没有硬件取消协议。若 Ctrl 仍为忙，软件不得仅清状态后覆盖任务参数。

手册给出轮询和中断两种执行方式；当前 TinySoC 使用轮询。FrozenVTA 对可选 interrupt 输出不连接 CPU IRQ，且没有配套 NPU ISR，因此中断模式不属于当前已实现能力。

5.5 用例与结果输出

Case	调用路径	结果检查与约束
dense_gemm_bias_clip	vta_case_run	检查 golden 覆盖的输出范围；适配本地 16×16 配置。
mlperf_resnet8_qconv	vta_case_run	检查卷积算子输出；其名称不代表完整 ResNet 模型验收。
bc_npu	外部 npu_run	当前入口比较 2 个 int8 输出；模型与数据头另行交付。

bc_npu 未复用通用 vta_case_run 的启动和超时流程。其入口存在首个输入值为 0 时直接结束的分支，且该分支未打印 UART 判定。该行为不能作为正常 NPU 计算通过的证据；外部 npu_run 的缓存与超时处理应随模型代码独立核对。

软件结果	数值	含义
VTA_CASE_OK	0	通用任务执行完成。
VTA_CASE_ERR_ARGUMENT	-1	任务参数不满足基本条件。
VTA_CASE_ERR_BUSY	-2	Ctrl START 仍为 1。
VTA_CASE_ERR_TIMEOUT	-3	达到轮询上限。
vta_case_result	0x56544100 0x56544150 0x56544146	依次为未运行、PASS 和 FAIL。

宿主 run_case_firmware.py 读取 UART 输出，检测到 VTA case PASS 返回 0，检测到 VTA case FAIL 返回 1，并终止模拟器。模拟器无判定退出时按失败处理。当前宿主运行器没有全局运行时限；任务挂起可能持续占用仿真进程。

5.6 构建与验证规则

make rtl 同步 RTL 哈希和 manifest，再用 Verilator 提取顶层端口并生成 interface.json；它不运行 Chisel 生成器。make generate 生成当前仿真平台的 SoC 工程，make sim 编译指定用例固件与模拟器，make run 执行已构建的模拟器。

make run 不重新构建模拟器，也不执行配置或 RTL 检查。修改 RTL、配置、固件或用例后，必须先重新构建，避免运行旧镜像。构建产物按 case 名命名，切换不同配置时应使用独立 BUILD_DIR。

检查层次	当前覆盖与判据
配置和包校验	检查地址重叠、容量约束、协议、RTL / 配置 / 接口哈希与顶层一致性。
单元测试	现有测试涉及地址映射、160 KiB 精确译码、端口识别和额外端口处理。
RTL lint	本地 VTA 以 Verilator 4.028 检查，0 错误、163 告警；完整清单因正式封装缺项未完成。
SoC 功能仿真	编译和运行后须出现真实执行路径的 PASS，并记录配置、数据版本和输出比较范围。
CDC / RDC	当前仅完成单时钟连接的结构检查，没有专业工具签核结论。
综合与 FPGA	当前未提供硬件 target、约束、PPA 结果或 bitstream 上板验收。

lint 无错误不能证明 CDC 安全、协议正确或数据计算正确。功能 PASS 也不表示所有张量形状、异常路径和压力场景均已覆盖。新增 IP、改变位宽或调整 burst 时，应补充背压、边界、错误响应、连续启动及复位场景。

5.7 移植与启动规则

软件环境基线为 Python 3.10、LiteX / LiteDRAM 2024.12 和 Migen 0.9.2，外加 Verilator、RISC-V 裸机工具链及宿主 C/C++ 编译器。Python 依赖和工具链分别离线分发，.venv 应在目标机器重建，不能依赖复制后的绝对 shebang。

服务器使用 csh / tcsh。项目根目录由 TINYVTA_ROOT 或 VTA_PATH 推导；TINYVTA_ROOT=$VTA_PATH/soc，实际 SoC Makefile 位于 $TINYVTA_ROOT/soc。离线包、tools、gcc-runtime 和 wheelhouse 默认放在 $HOME，tools 与 gcc-runtime 须保持同级。

Flash / FPGA 启动必须替换仿真预加载机制：先建立有效时钟与复位，初始化目标内存，将固件和张量数据装入最终地址，再启动 NPU。若 LMA 与 VMA 不同，应明确由引导程序、主机下载或 DMA 完成装载，并同步修改当前禁止 CPU 数组复制的链接约定。

附录 A 参考文件与运行入口

文档内容对应以下参考版本和源码。文件路径以 tinysoc_wangshu 根目录为基准；手册页码为 PDF 页序。

编号	参考文件	适用内容
1	20250715 xdl - NPU User Guide V1.1-1.pdf	第 1 至 2 页模块与能力；第 3 至 6 页寄存器；第 6 至 8 页执行模式。
2	CPB_Spec(2).docx，版本 1.0.3	章节组织、封面、修订记录、页眉和位域表形式。
3	soc/config/ty1b_simple_npu.json	SoC 配置与地址空间。
4	generated/vta-ip/apb32-axi64/rtl/vta.v	本地 VCR、计数器、VTA 顶层和计算实现。
5	soc/litex/ty1b_soc.py 与 vta_ip.py	SoC、总线桥、时钟复位、可选端口和中断连接。
6	soc/litex/axi_burst_splitter.py	读写片段、完成与错误响应合并。
7	soc/linker 与 soc/firmware	存储布局、镜像提取和软件运行时。
8	soc/Makefile 与 soc/tools	构建入口、元数据校验、接口生成和运行判定。

A.1 服务器环境与执行

```
setenv VTA_PATH /path/to/vta_project
cd "$VTA_PATH/soc"
./setup_env.csh
source tinyvta_env.csh
cd "$TINYVTA_ROOT/soc"
make rtl
make rtl-check
make interface-test
make sim CASE=bc_npu
make run CASE=bc_npu
```

上述 BC 流程的前提是正式 RTL、六个 mem_wrap 封装和五个 BC 数据头已放入包内，并完成 VTA_BADDR 地址适配。首次识别正式顶层的时钟和 PSELx 时，可显式指定下列覆盖；覆盖会保存在 interface.json 中。

```
make rtl \
  INTERFACE_ARGS="--map clock=clock --map host.psel=io_host_PSELx"
```

若只运行本地 16×16 用例，应先为该 RTL 提供完整且匹配的文件清单，再选择 dense_gemm_bias_clip 或 mlperf_resnet8_qconv。不要把当前清版占位清单直接当作已闭合 IP 包。

附录 B 配置差异与扩展边界

项目	当前定义或差异	发布与移植要求
8×8 与 16×16	手册和服务器目标为 8×8；本地 RTL 为 16×16。	分别发布配置、指令与数据，禁止混用。
内部 Buffer 容量	本地配置文件记录 LOG_* 参数，正式容量依 RTL。	逐项核对实际数组 / 宏深度，不仅检查哈希。
AccWrCnt	手册标 W，本地实现为完成统计快照并可写。	正式 ASIC 需确认实际访问和更新语义。
中断模式	手册描述；当前 CPU 未接 NPU interrupt。	增加 IRQ、ISR、清除规则与回归后才能启用。
ASIC SRAM 封装	清版包只有预留路径；正式文件外部交付。	连同宏模型及综合视图管理。
FPGA 内存	仿真数组、DDR 模型及预加载不能直接代表硬件。	实现 BRAM、真实 DDR 与镜像装载。
多时钟	当前 clock / host.pclk 绑定同一 sys。	增加异步桥、同步器和 CDC / RDC 约束。
容量和位宽	校验器有固定容量；部分接口允许更多宽度。	同时修改总线、拆分限制、链接与测试。
超时与恢复	软件有轮询次数上限，宿主无全局时限。	按目标要求增加总线 / 宿主超时和复位恢复。

B.1 当前本地冻结包标识

顶层 VTA_Top；控制协议 APB4 32 bit；DMA 数据 64 bit，地址 32 bit，ID 8 bit。以下哈希用于识别本地基线，不代替对正式 ASIC 包的核对。

RTL SHA256

```
c63e432318179bd8e8ef8192a6954d20a1c0664274f226b4103ccb2cb31b4771
```

VTA Config SHA256

```
6876e92fabe0141a7fdbbe6762889d81f9512fcfbc7d45d6dc1919f639905180
```

Interface SHA256

```
289fcf723746ecff4d2f79924233cc35a63d96c06a4cac802dfd22731b7e5220
```